

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
SUL-RIO-GRANDENSE
CÂMPUS PASSO FUNDO**

CURSO: Ciência da Computação

DISCIPLINA: Estrutura de Dados III

PROFESSOR: Leonardo Deliyannis Constantin

ALUNOS(AS): Daniel Cipriano da Silva, Keila de Sales Gonçalves e Maria Elvira Trojan

HEAPSORT

O algoritmo **heapsort** é um algoritmo de ordenação comparativo de propósito geral, e faz parte da família de algoritmos de ordenação por seleção (Selection Sort).

Enquanto a seleção tradicional gasta tempo linear $O(n)$ a cada iteração para varrer o vetor procurando o maior elemento, o Heapsort otimiza drasticamente essa etapa ao organizar os dados em uma estrutura de Heap Binário, permitindo consultar e extrair elementos extremos em tempo logarítmico.

1. Estrutura do Heap Binário e a Representação em Vetor

Conceitualmente, um heap binário é visualizado como uma **árvore binária quase completa** (todos os níveis estão preenchidos, com exceção eventual do último, que se preenche da esquerda para a direita). Na prática, ele dispensa ponteiros e nós dinâmicos: **é mapeado diretamente em um vetor**.

Para qualquer elemento no índice i de um vetor de tamanho n (indexado em 0):

- **Índice do Pai:** $\lfloor (i - 1) / 2 \rfloor$
- **Filho Esquerdo:** $2i + 1$
- **Filho Direito:** $2i + 2$

- **Nós Folha:** Todas as posições de $\lfloor n/2 \rfloor$ até $n-1$ são folhas e satisfazem trivialmente as condições de heap.

Variações do Heap

- **Max-Heap (Heap Máximo):** Cada nó pai é maior ou igual aos seus filhos ($A[pai(i)] \geq A[i]$). O maior elemento do conjunto reside na raiz ($A[0]$). É utilizado para ordenar em ordem crescente.
- **Min-Heap (Heap Mínimo):** Cada nó pai é menor ou igual aos seus filhos ($A[pai(i)] \leq A[i]$). O menor elemento reside na raiz. É utilizado para ordenar em ordem decrescente.

2. Mecânica do Algoritmo

O Heapsort opera estritamente no próprio vetor e segue dois passos fundamentais:

Construção do Heap Inicial (buildHeap):

- Transforma o array desordenado em um Max-Heap válido.
- O processo ocorre de baixo para cima (*bottom-up*), iniciando no último nó pai ($i = (n-1)/2$) e retrocedendo até a raiz ($i = 0$), aplicando a rotina auxiliar **heapify** em cada nó para afundar os elementos fora de posição.
- Embora pareça levar mais tempo, essa etapa tem custo linear de apenas $O(n)$ (algoritmo linear de Floyd), já que a maior parte dos nós está nas camadas mais baixas e desce alturas muito pequenas.

Ordenação (Extração e Reconstrução):

- Com o Max-Heap montado, o maior elemento estará sempre na raiz da árvore (posição 0 do array).
- Esse elemento da raiz é trocado com o último elemento útil do array, passando a ocupar sua posição definitiva no final.
- O tamanho útil do heap é reduzido em 1 e a função **heapify** é chamada sobre a nova raiz para restaurar a propriedade de Max-Heap (operação de custo $O(\log n)$).
- Esse ciclo de troca e restauração repete-se $n-1$ vezes até que todos os elementos estejam em suas posições corretas.

heapify é o processo de transformar uma lista ou árvore binária desordenada em uma estrutura de dados do tipo **heap**, ajustando a posição dos elementos para que eles sigam a regra de ordem da árvore.

3. Complexidade de Tempo e Espaço

Complexidade de Tempo: É $O(n \log n)$ (ou $\Theta(n \log n)$) para o melhor caso, pior caso e caso médio.

- A construção inicial do heap consome $O(n)$ operações.
- A fase de ordenação executa repetidas remoções do elemento máximo com reconstrução do heap ($O(\log n)$). Como essa restauração ocorre $n - 1$ vezes, a soma total resulta em $O(n) + O(n \log n) = O(n \log n)$.

Complexidade de Espaço: É $O(1)$ (memória auxiliar constante). Ele é um algoritmo *in-place*, realizando todas as permutações diretamente no próprio vetor de entrada, sem a necessidade de arranjos temporários.

4. Estabilidade

- **Não é um algoritmo estável.**
- As permutações realizadas durante o heapify e a troca da raiz com o último elemento movem chaves por saltos longos na árvore, não preservando a ordem relativa original de elementos com valores idênticos.

5. Vantagens e Desvantagens

Vantagens

- **Desempenho Estável e Consistente:** O tempo de execução é determinístico e não depende da disposição inicial dos dados. Diferente do Quicksort (que pode degradar para $O(n^2)$ no pior caso), o Heapsort garante sempre $O(n \log n)$.
- **Eficiência de Memória:** Opera com custo de memória $O(1)$, sendo superior ao Merge Sort (que exige $O(n)$ de memória auxiliar). É ideal para ordenar grandes conjuntos de dados quando a memória principal é restrita.

Desvantagens

- **Localidade de Referência Fraca:** Ao percorrer a árvore através dos saltos de índices ($2i + 1$), o algoritmo acessa posições não contíguas na memória física, gerando mais falhas de cache (*cache misses*) na CPU se comparado ao Quicksort.
- **Não Preserva a Estabilidade:** Inadequado caso seja obrigatório manter a ordenação prévia de múltiplos critérios de ordenação.

cache misses é quando um sistema ou processador procura um dado na memória cache, mas não o encontra lá.

- Vídeos no Youtube:

[HEAP-sort with Hungarian folk dance](#)
[Funcionamento do HeapSort](#)
[Heap Sort - Algorithms in 60 Seconds](#)

6. Materiais e Leituras Complementares Recomendadas

Leitura:

CORMEN, T. H. et al. *Algoritmos: Teoria e Prática* (4ª Edição). **Capítulo 6: Heapsort** (páginas 151 a 169).

- *Por que ler:* É a principal fonte teórica de onde saem as questões de concurso (POSCOMP e ENADE), cobrindo a prova matemática da construção linear $O(n)$ do heap e o funcionamento de filas de prioridade.

CALAZANS, Luiz. [Entendendo o Heapsort](#). Publicado na plataforma **Medium**.

- *Por que ler:* Explicação clara e acessível sobre a mecânica do algoritmo com ilustrações passo a passo, detalhando a complexidade temporal $O(n \log n)$ e a execução *in-place*.

Simulador Visual Interativo:

VisuAlgo (Binary Heap / Heapsort): <https://visualgo.net/en/heap>

- *Por que usar:* Permite inserir números arbitrários e visualizar o vetor sendo transformado em árvore, acompanhando passo a passo as trocas da função `heapify` e a extração da raiz.

Videoaulas:

Canal: *Programação Descomplicada* (Prof. André Backes) — [Estrutura de Dados em C | Aula 53 - Ordenação - HeapSort](#).

- *Por que assistir:* Explica de forma visual e intuitiva o mapeamento do vetor em árvore binária e a lógica das fórmulas dos índices em linguagem C.

Canal: *podcast KING* — [HEAP-sort with Hungarian folk dance](#).

- *Por que assistir:* Demonstração visual clássica onde dançarinos representam nós do arranjo, encenando as trocas do *heapify* e a migração da raiz para o fim do vetor.

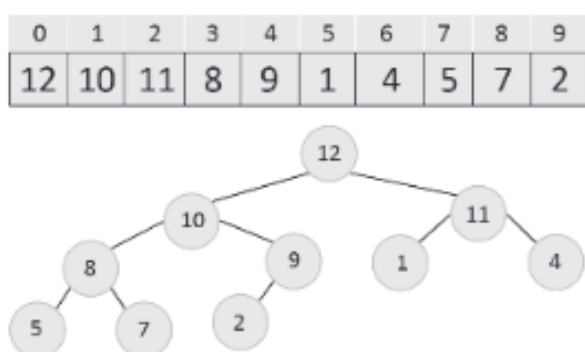
Canal: *OpenAMind* — [Heap Sort - Algorithms in 60 Seconds](#).

- *Por que assistir:* Revisão relâmpago de 1 minuto que sintetiza as etapas essenciais: montagem da árvore, rotina de *heapify* e extração ordenada do topo.

7. Questões de Prova Resolvidas e Comentadas

ENADE 2021- QUESTÃO DISCURSIVA 05

Um *heap* binário é um arranjo que pode ser visualizado como uma árvore binária, sendo que cada nó da árvore corresponde a um elemento do arranjo, como pode ser observado na figura a seguir.



Percebe-se que existem dois tipos de *heaps*: *heaps* máximo e *heaps* mínimo. O *heap* máximo é uma estrutura de dados que possibilita a consulta ou extração de forma eficiente do maior elemento de uma coleção. A propriedade de *heap* máximo especifica que um nó filho (no código calculado pelas funções *left* e *right*) tem sempre armazenado um valor menor ou igual ao seu pai.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 3. ed. MIT Press and McGraw-Hill, p. 131-161, 2009 (adaptado).

Considerando a implementação a seguir, o *heapify* é uma função auxiliar para reorganizar o arranjo (garantindo a propriedade de *heap* máximo em uma determinada posição do arranjo) e *buildHeap* é uma função que usa *heapify* para reorganizar todas as posições do arranjo (garantindo a propriedade de *heap* máximo para todos os elementos).

```
int left(int i) { return (2 * i + 1); }
int right(int i) { return (2 * i + 2); }

/* a - arranjo, n - número de elementos,
   i - posição do elemento que deve ser
   colocado em propriedade de heap */

void heapify(int *a, int n, int i)
{
    int e, d, max, aux;

    e = left(i);
    d = right(i);
    if (e < n && a[e] > a[i])
        max = e;
    else
        max = i;
    if (d < n && a[d] > a[max])
        max = d;
    if (max != i)
    {
        aux = a[i];
        a[i] = a[max];
        a[max] = aux;
        heapify(a, n, max);
    }
}

/* a - arranjo, n - número de elementos */
void buildHeap(int *a, int n)
{
    int i;
    for (i = (n - 1) / 2; i >= 0; i--)
        heapify(a, n, i);
}
```

De acordo com as informações apresentadas, faça o que se pede no item a seguir.

a) Apresente a complexidade de tempo no pior caso para a função *heapify*, use a notação O ou Θ .

ENADE 2011- QUESTÃO 24

As filas de prioridades (*heaps*) são estruturas de dados importantes no projeto de algoritmos. Em especial, *heaps* podem ser utilizados na recuperação de informação em grandes bases de dados constituídos por textos. Basicamente, para se exibir o resultado de uma consulta, os documentos recuperados são ordenados de acordo com a relevância presumida para o usuário. Uma consulta pode recuperar milhões de documentos que certamente não serão todos examinados. Na verdade, o usuário examina os primeiros m documentos dos n recuperados, em que m é da ordem de algumas dezenas.

Considerando as características dos *heaps* e sua aplicação no problema descrito acima, avalie as seguintes afirmações.

- I. Uma vez que o *heap* é implementado como uma árvore binária de pesquisa essencialmente completa, o custo computacional para sua construção é $O(n \log n)$.
- II. A implementação de *heaps* utilizando-se vetores é eficiente em tempo de execução e em espaço de armazenamento, pois o pai de um elemento armazenado na posição i se encontra armazenado na posição $2i + 1$.
- III. O custo computacional para se recuperar de forma ordenada os m documentos mais relevantes armazenados em um *heap* de tamanho n é $O(m \log n)$.
- IV. Determinar o documento com maior valor de relevância armazenado em um *heap* tem custo computacional $O(1)$.

Está correto apenas o que se afirma em:

- A) I e II.
- B) II e III.
- C) III e IV.
- D) I, II e IV.
- E) I, III e IV.

Gabarito e Fundamentação

Questão 1 (Questão 05 – ENADE 2021)

A complexidade de tempo no pior caso para a função *heapify* é $O(\log n)$ ou $\Theta(\log n)$.

- **Justificação:** A função percorre um único caminho descendente da raiz até uma das folhas da árvore binária quase completa. Como a altura de uma árvore binária quase completa contendo n elementos é dada por $h = \lfloor \log_2 n \rfloor$, o número máximo de chamadas recursivas e comparações é proporcional à altura da árvore, resultando em um custo assintótico de $O(\log n)$ (ou $\Theta(\log n)$ em função da altura h da subárvore).

Questão 2 (Questão 24 – ENADE 2011)

- **Item I (Incorreto):** O *heap* binário segue a propriedade de ordenação de *heap* (relação de ordem entre pai e filhos diretos) e não a propriedade de uma árvore binária de pesquisa (onde subárvores à esquerda são menores e à direita maiores). Adicionalmente, a construção clássica (*build-heap*) é realizada em tempo linear $O(n)$ e não obrigatoriamente $O(n \log n)$.
- **Item II (Incorreto):** A fórmula $2i + 1$ calcula a posição de um dos **filhos** do nó i (quando a indexação se inicia em zero). O índice do pai encontra-se em $\lfloor (i - 1) / 2 \rfloor$.
- **Item III (Correto):** Cada operação de remoção da raiz combinada com o restabelecimento da propriedade de *heap* leva tempo $O(\log n)$. Para m extrações ordenadas, o custo total atinge $O(m \log n)$.
- **Item IV (Correto):** O maior elemento encontra-se permanentemente na raiz (primeira posição do arranjo), exigindo apenas acesso direto de tempo constante $O(1)$.

Estrutura de Dados Heap Binária (Max-Heap)

```
#include <iostream>
#include <vector>
#include <utility>    // Para std::swap
#include <stdexcept> // Para runtime_error

using namespace std;

// Classe que encapsula o Binary Heap (Max-Heap) e o Heapsort
class MaxHeapBinaria {
private:
    vector<int> heap; // Vetor que armazena os nós da árvore quase completa
    int tamanho;      // Quantidade de elementos atualmente no heap
    int capacidade;    // Capacidade máxima reservada

    // Procedimento para manter a propriedade descendo o elemento (O(log n))
    void heapify(int indice) {
        int maior = indice;
        int filho_esquerda = 2 * indice + 1; // Índice do filho à esquerda (base 0)
        int filho_direita = 2 * indice + 2; // Índice do filho à direita (base 0)

        // Compara com o filho da esquerda: procura se ele é MAIOR
        if (filho_esquerda < tamanho && heap[filho_esquerda] > heap[maior]) {
            maior = filho_esquerda;
        }

        // Compara com o filho da direita: procura se ele é MAIOR
        if (filho_direita < tamanho && heap[filho_direita] > heap[maior]) {
            maior = filho_direita;
        }

        // Se o maior elemento não for o próprio índice, troca e desce recursivamente
        if (maior != indice) {
            trocar(indice, maior);
            heapify(maior);
        }
    }

    // Função utilitária para trocar dois elementos de posição no arranjo
    void trocar(int i, int j) {
        swap(heap[i], heap[j]);
    }
};
```

```
}

public:
    // Construtor: aloca o espaço com base na capacidade informada
    MaxHeapBinaria(int cap) {
        capacidade = cap;
        tamanho = 0;
        heap.resize(capacidade);
    }

    // Retorna a quantidade de elementos válidos no heap
    int obter_tamanho() const {
        return tamanho;
    }

    // Insere um novo elemento no Heap (O(log n))
    void inserir(int valor) {
        if (tamanho == capacidade) {
            throw runtime_error("Erro: O Heap está cheio!");
        }

        // Adiciona o novo valor na primeira posição livre (final do heap)
        int indice_atual = tamanho;
        heap[indice_atual] = valor;
        tamanho++;

        // Flutua para cima: enquanto não for a raiz e for MAIOR que o pai
        while (indice_atual > 0 && heap[indice_atual] > heap[(indice_atual - 1) / 2]) {
            trocar(indice_atual, (indice_atual - 1) / 2);
            indice_atual = (indice_atual - 1) / 2; // Sobe para o índice do pai
        }
    }

    // Retorna o elemento da raiz sem remover (O(1))
    int obter_raiz() const {
        if (tamanho == 0) {
            throw runtime_error("Erro: O Heap está vazio!");
        }
        return heap[0];
    }

    // Remove e retorna o elemento da raiz (O(log n))
    int remover_raiz() {
```

```
if (tamanho == 0) {
    throw runtime_error("Erro: O Heap está vazio!");
}

int valor_raiz = heap[0]; // A raiz sempre armazena o maior valor

// Substitui a raiz pelo último elemento e reduz o tamanho
heap[0] = heap[tamanho - 1];
tamanho--;

// Restabelece a propriedade de Max-Heap a partir da nova raiz
heapify(0);

return valor_raiz;
}

// Algoritmo de Ordenação Heapsort (O(n log n)) -> Resulta em ordem CRESCENTE
void heap_sort() {
    // Passo 1: Constrói o Max-Heap (Build-Heap) de baixo para cima
    for (int i = tamanho / 2 - 1; i >= 0; i--) {
        heapify(i);
    }

    // Passo 2: Extrai a raiz sucessivamente para o final do arranjo
    for (int i = tamanho - 1; i >= 0; i--) {
        trocar(0, i);
        tamanho = i; // Isola o maior elemento já posicionado no final
        heapify(0);
    }
}

// Imprime os elementos no arranjo
void imprimir(int limite = -1) const {
    int n = (limite == -1) ? tamanho : limite;
    for (int i = 0; i < n; i++) {
        cout << heap[i] << " ";
    }
    cout << endl;
}

};

int main() {
```

```
vector<int> arranjo = { 7, 3, 9, 2, 4, 1, 5 };

cout << "=== DEMONSTRACAO DA MAX-HEAP ===" << endl;
cout << "Vetor original a ser inserido: ";
for (int num : arranjo) cout << num << " ";
cout << "\n\n";

// 1. Cria a Max-Heap e insere os elementos
MaxHeapBinaria max_heap(arranjo.size());
for (int num : arranjo) {
    max_heap.inserir(num);
}

cout << "Arranjo interno estruturado como Max-Heap: ";
max_heap.imprimir();

// 2. Consulta da raiz
cout << "Maior elemento atual (raiz): " << max_heap.obter_raiz() << endl;

// 3. Remocao do elemento de maior prioridade
cout << "Removendo o maior elemento: " << max_heap.remover_raiz() << endl;
cout << "Max-Heap apos a remocao: ";
max_heap.imprimir();

// 4. Teste de ordenacao completa via Heapsort
// Reinserindo o elemento para restaurar o vetor completo
max_heap.inserir(9);
cout << "\nExecutando Heapsort (ordem crescente)..." << endl;
max_heap.heap_sort();
cout << "Vetor totalmente ordenado: ";
max_heap.imprimir(arranjo.size());

return 0;
}
```

Min-Heap (Heap de Mínimo)

```
#include <iostream>
#include <vector>
#include <utility>    // Para std::swap
#include <stdexcept>  // Para runtime_error

using namespace std;

// Classe que encapsula o Binary Heap (Min-Heap) e o Heapsort
class MinHeapBinaria {
private:
    vector<int> heap; // Vetor que armazena os nós da árvore quase completa
    int tamanho;      // Quantidade de elementos atualmente no heap
    int capacidade;   // Capacidade máxima reservada

    // Procedimento para manter a propriedade descendo o elemento (O(log n))
    void heapify(int indice) {
        int menor = indice;
        int filho_esquerda = 2 * indice + 1; // Índice do filho à esquerda (base 0)
        int filho_direita = 2 * indice + 2; // Índice do filho à direita (base 0)

        // Compara com o filho da esquerda: procura se ele é MENOR
        if (filho_esquerda < tamanho && heap[filho_esquerda] < heap[menor]) {
            menor = filho_esquerda;
        }

        // Compara com o filho da direita: procura se ele é MENOR
        if (filho_direita < tamanho && heap[filho_direita] < heap[menor]) {
            menor = filho_direita;
        }

        // Se o menor elemento não for o próprio índice, troca e desce recursivamente
        if (menor != indice) {
            trocar(indice, menor);
            heapify(menor);
        }
    }

    // Função utilitária para trocar dois elementos de posição no arranjo
    void trocar(int i, int j) {
        swap(heap[i], heap[j]);
    }
}
```

```
public:
    // Construtor: aloca o espaço com base na capacidade informada
    MinHeapBinaria(int cap) {
        capacidade = cap;
        tamanho = 0;
        heap.resize(capacidade);
    }

    // Retorna a quantidade de elementos válidos no heap
    int obter_tamanho() const {
        return tamanho;
    }

    // Insere um novo elemento no Heap (O(log n))
    void inserir(int valor) {
        if (tamanho == capacidade) {
            throw runtime_error("Erro: O Heap está cheio!");
        }

        // Adiciona o novo valor na primeira posição livre (final do heap)
        int indice_atual = tamanho;
        heap[indice_atual] = valor;
        tamanho++;

        // Flutua para cima: enquanto não for a raiz e for MENOR que o pai
        while (indice_atual > 0 && heap[indice_atual] < heap[(indice_atual - 1) / 2]) {
            trocar(indice_atual, (indice_atual - 1) / 2);
            indice_atual = (indice_atual - 1) / 2; // Sobe para o índice do pai
        }
    }

    // Retorna o elemento da raiz sem remover (O(1))
    int obter_raiz() const {
        if (tamanho == 0) {
            throw runtime_error("Erro: O Heap está vazio!");
        }
        return heap[0];
    }

    // Remove e retorna o elemento da raiz (O(log n))
    int remover_raiz() {
        if (tamanho == 0) {
```

```
        throw runtime_error("Erro: O Heap está vazio!");
    }

    int valor_raiz = heap[0]; // A raiz sempre armazena o menor valor

    // Substitui a raiz pelo último elemento e reduz o tamanho
    heap[0] = heap[tamanho - 1];
    tamanho--;

    // Restabelece a propriedade de Min-Heap a partir da nova raiz
    heapify(0);

    return valor_raiz;
}

// Algoritmo de Ordenação Heapsort (O(n log n)) -> Resulta em ordem DECRESCENTE
void heap_sort() {
    // Passo 1: Constrói o Min-Heap (Build-Heap) de baixo para cima
    for (int i = tamanho / 2 - 1; i >= 0; i--) {
        heapify(i);
    }

    // Passo 2: Extrai a raiz sucessivamente para o final do arranjo
    for (int i = tamanho - 1; i >= 0; i--) {
        trocar(0, i);
        tamanho = i; // Isola o menor elemento já posicionado no final
        heapify(0);
    }
}

// Imprime os elementos no arranjo
void imprimir(int limite = -1) const {
    int n = (limite == -1) ? tamanho : limite;
    for (int i = 0; i < n; i++) {
        cout << heap[i] << " ";
    }
    cout << endl;
}

};

int main() {
    vector<int> arranjo = { 7, 3, 9, 2, 4, 1, 5 };
}
```

```
cout << "=== DEMONSTRACAO DA MIN-HEAP ===" << endl;
cout << "Vetor original a ser inserido: ";
for (int num : arranjo) cout << num << " ";
cout << "\n\n";

// 1. Cria a Min-Heap e insere os elementos
MinHeapBinaria min_heap(arranjo.size());
for (int num : arranjo) {
    min_heap.inserir(num);
}

cout << "Arranjo interno estruturado como Min-Heap: ";
min_heap.imprimir();

// 2. Consulta da raiz
cout << "Menor elemento atual (raiz): " << min_heap.obter_raiz() << endl;

// 3. Remocao do elemento de menor prioridade
cout << "Removendo o menor elemento: " << min_heap.remover_raiz() << endl;
cout << "Min-Heap apos a remocao: ";
min_heap.imprimir();

// 4. Teste de ordenacao completa via Heapsort
// Reinserindo o elemento para restaurar o vetor completo
min_heap.inserir(1);
cout << "\nExecutando Heapsort (ordem decrescente)..." << endl;
min_heap.heap_sort();
cout << "Vetor totalmente ordenado: ";
min_heap.imprimir(arranjo.size());

return 0;
}
```